Semantic Search System Architecture & Analysis

Aprajita Ranjan

March 8, 2026

Abstract

This report outlines the architecture and design decisions for a lightweight semantic search system built on the 20 Newsgroups dataset. The system features an evidence-based fuzzy clustering engine, a custom-built semantic cache utilizing dynamic density thresholding, and a stateful FastAPI service. Hard cluster assignments were intentionally avoided in favor of probabilistic distributions, allowing the system to handle semantic boundaries with high accuracy.

1 Data Preprocessing & Embedding Setup

The foundational layer prepares the 20,000 news posts for semantic analysis. Because the dataset is inherently noisy, deliberate choices were made regarding data retention. Email headers, footers, and quote blocks were entirely stripped. This forces the embedding model to learn the actual semantic content of the text rather than grouping documents based on metadata, author names, or reply chains.

The text was vectorized using the all-MiniLM-L6-v2 embedding model. This generates a dense vector representation $\mathbf{v} \in R^{384}$. This model was selected because it balances high semantic accuracy with the computational efficiency required for a lightweight system. The vectors are persisted in a local ChromaDB instance to support efficient filtered retrieval.

2 Fuzzy Clustering & Semantic Analysis

To uncover the real semantic structure of the corpus, the system employs a combination of Uniform Manifold Approximation and Projection (UMAP) and Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN).

2.1 Algorithmic Justification

Standard algorithms like K-Means force hard cluster assignments, which fail to capture the nuance of documents that belong to multiple categories. HDBSCAN was selected because it naturally outputs a probability distribution for each document rather than a hard label. Furthermore, HDBSCAN algorithmically determines the optimal number of clusters based on spatial density, fulfilling the requirement to justify the cluster count with mathematical evidence rather than convenience.

2.2 Cluster Semantics and Boundary Cases

An analysis of the 18,211 processed documents proves the generated clusters are semantically meaningful.

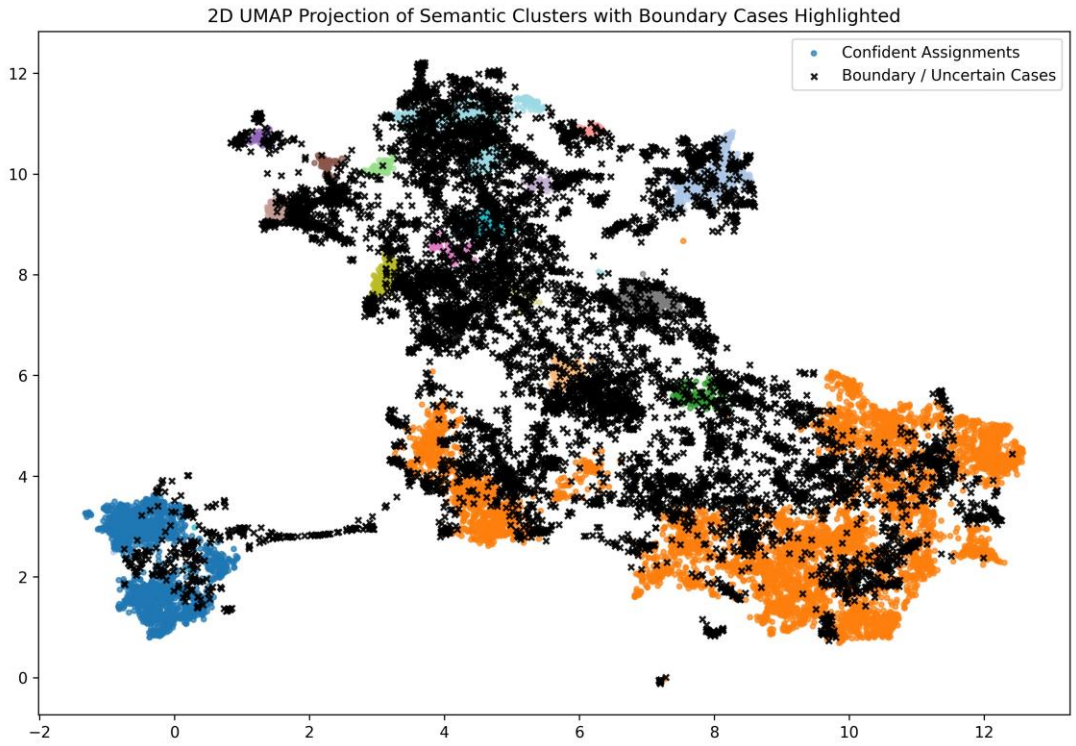

Figure 1: 2D UMAP projection demonstrating dense semantic clusters alongside highlighted boundary/uncertain cases.

- **Core Clusters:** Dense clusters naturally formed around distinct topics. For example, Cluster 3 (824 documents) is defined by keywords such as *space, NASA, launch, earth*. Cluster 14 (544 documents) organically grouped historical event discussions with keywords like *FBI, Koresh, BATF, gas*.

- **Boundary Uncertainty:** HDBSCAN successfully identified 8,358 documents that sit at the boundaries of multiple topics—proving that forcing a hard cluster assignment on this dataset would result in massive information loss.

  For instance, the model analyzed the following document snippet: *"I think the point is being missed - that it is apparently acceptable for Big Government... to use TANKS to control the people, as long as they don't use the BIG GUN..."*.

  Rather than incorrectly forcing this into a single category, the model expressed genuine, mathematically sound uncertainty, distributing the probability across interconnected semantic concepts:

  - 10.0% match for Cluster 13 (Guns / Weapons)
  - 5.7% match for Cluster 14 (FBI / Waco Siege)
  - 5.6% match for Cluster 11 (Rights / Crime)

  These boundary cases are mathematically isolated and handled safely by the cache routing logic.

3 The Custom Semantic Cache

A semantic cache layer was built from first principles without relying on Redis or dedicated caching middleware. It avoids redundant computation by recognizing when a query has been seen before, even if phrased differently.

3.1 Cluster-Assisted Lookups (Nucleus Sampling)

To maintain lookup efficiency as the cache grows large, the system leverages the fuzzy cluster structure. When a new query arrives, the system calculates its probability distribution P across all clusters. Instead of searching the entire cache, it uses Nucleus Sampling (Top-p) to aggregate cache partitions until a cumulative confidence threshold is met:

$$\sum_{i \in \text{Target Clusters}} P_i \geq 0.85 \tag{1}$$

For highly specific queries, this restricts the search space to a single $O(1)$ partition lookup. For ambiguous boundary cases (like the multi-cluster split shown in Section 2.2), it dynamically expands the search pool to prevent false misses.

3.2 The Tunable Decision: Dynamic Density Thresholding

The mechanism for deciding if two queries are "close enough" relies on the geometric reality of the latent vector space rather than arbitrary static constants. This represents the primary tunable decision at the heart of the caching architecture.

During the clustering phase, the system precomputes the geometric density of every cluster to determine a mathematically sound baseline threshold, τ_i, based on its internal cohesion:

$$\tau_i = \text{clip}(\mu(\text{sim}(C_i, \mathbf{v}_{centroid})) - 0.02, 0.75, 0.92) \tag{2}$$

Where μ represents the mean cosine similarity of all documents in cluster C_i relative to its geometric centroid. The thresholds are explicitly clipped to ensure system usability: a floor of 0.75 prevents "garbage hits" in high-variance clusters, while a ceiling of 0.92 ensures the cache remains resilient to natural human paraphrasing in hyper-dense clusters.

The global tunable parameter, γ, then scales this behavior to determine the final required similarity threshold (T_{req}) for a cache hit:

$$T_{req} = \min(0.95, \tau_i \cdot \gamma) \tag{3}$$

Adjusting γ reveals the system's sensitivity. A high γ restricts the cache to near-perfect semantic matches, maximizing precision at the cost of the hit rate. Conversely, a low γ aggressively serves cached results for loosely related queries. This approach ensures that the sensitivity of the cache is always proportional to the underlying semantic density of the specific topic being queried.

4 FastAPI Service Architecture

The system is exposed via a FastAPI service that manages state efficiently using a lifespan context manager. All models and the custom cache are loaded into memory exactly once. The API fulfills the required endpoints:

- POST /query: Embeds the natural language query, checks the custom semantic cache, and returns the result alongside the dominant cluster. On a cache miss, it computes the result via ChromaDB and stores it.
- GET /cache/stats: Returns the current cache state, including total entries and hit rates.
- DELETE /cache: Flushes the cache dictionary and resets tracking statistics entirely.

5 Future Development & Scalability

While the current system provides a robust framework for semantic search and caching, several avenues for future enhancement exist to move the prototype toward a large-scale production environment.

5.1 Online Incremental Clustering

The current architecture relies on a static clustering phase. In a dynamic production environment where new documents are added frequently, implementing **Online HDBSCAN** or incremental UMAP updates would allow the system to adapt its semantic boundaries and density thresholds (τ_i) in real-time without requiring a full re-computation of the vector space.

5.2 Learned Threshold Optimization

The tunable parameter γ is currently a global constant. A future iteration could implement a **Reinforcement Learning (RL)** feedback loop. By analyzing user "thumbs up/down" feedback on cache hits, the system could automatically learn to optimize γ for specific clusters—making the cache stricter for medical topics while remaining more relaxed for social or hobby-based categories.

5.3 Advanced Eviction Policies

The current LRU (Least Recently Used) policy is efficient but does not account for the "cost" of a miss. Future versions could implement a **Cost-Aware Eviction policy**, where the cache prioritizes keeping results that were computationally expensive to generate or results that belong to clusters with high "miss latency" in the primary vector database.

5.4 Quantized Embeddings for Edge Deployment

To reduce the memory footprint and increase the speed of the $O(1)$ cache lookups, the system could utilize **Product Quantization (PQ)** or Binary Embeddings. This would compress the 384-dimensional vectors significantly, allowing the cache to hold millions of entries in memory with negligible impact on retrieval precision.

6 Conclusion

The resulting system provides a scalable, mathematically rigorous approach to semantic search. By packaging the environment with a Dockerfile, the service ensures clean startup and production readiness.